

锂离子电池快充策略对寿命衰减的影响建模与优化

摘 要

提高充电倍率可缩短补能时间,但过高电流会加速容量衰减。本文基于 MIT-Stanford 公开的 49 块 A123 18650 型磷酸铁锂电池循环老化数据,围绕两阶段快充策略 $C_1-Q_1-C_2$, 研究快充参数对寿命衰减的影响,并讨论寿命预测与策略优化问题。

针对问题一,本文对循环数据进行逐电池、逐字段的 IQR 异常检测与前后邻域均值填补,共替换 559 个异常单元格;将原始 9 种策略字符串按充电配方统一为 7 类,并以第 200 次循环的 SOH 作为观测窗口内的寿命代理指标(数据尚未覆盖 $\text{SOH} = 80\%$ 的真实寿命终止)。在 40 块满 200 圈电池中,SOH 均值为 0.9914。若仅按策略 SOH 中位数,合并后的 4.8C(80%) – 4.8C 最高,但组内离散过大、充电时间还分裂为两簇,故不以该组为典型。组内更集中的典型长寿命策略为 5C(67%) – 4C、5.3C(54%) – 4C、5.6C(36%) – 4.3C 与 5.6C(19%) – 4.6C,共同特征是 $C_1 > C_2$,充电时间约 10.04 min;典型短寿命策略为 3.7C(31%) – 5.9C,特征为 $C_1 < C_2$,在相近充电时间下 SOH 明显更低。结果表明:高 SOC 区间维持高倍率更伤寿命;提高电流时适当降低切换 SOC Q_1 有助于在几乎不增加充电时间的前提下保持健康状态。温度、内阻与初始容量不能单独解释这一长短寿差异。

针对问题二,拟在问题一基础上检验策略间寿命差异的统计显著性,分析 C_1, Q_1, C_2 与寿命代理指标的关系,并讨论不同 SOC 区间高倍率充电的衰减差异及机制局限。(本节定量结果待补充)

针对问题三,拟利用截至第 150 次循环的早期数据,提取健康特征并建立 SOH 衰减预测模型,预报第 151–200 次循环 SOH,并外推至 $\text{SOH} = 80\%$ 的循环寿命;同时比较不同早期窗口长度及策略信息的作用。(本节定量结果待补充)

针对问题四,拟建立充电时间模型与寿命衰减模型,在已有实验策略及其合理邻域内进行充电时间–寿命的多目标权衡,给出推荐策略并讨论外推风险。(本节定量结果待补充)

关键词: 锂离子电池; 快充策略; SOH; 循环寿命; IQR; 寿命预测; 多目标优化

1 问题重述

随着新能源汽车与储能系统对补能效率的要求提高，锂离子电池快充成为缩短充电时间的主要手段。然而较高充电电流会增强极化、加剧热效应并加速副反应，从而加快容量衰减、缩短循环寿命 [1–3]。电池健康状态定义为

$$\text{SOH} = \frac{Q_{\text{current}}}{Q_{\text{initial}}}, \quad (1)$$

本题以 $\text{SOH} = 80\%$ 作为寿命终止 (EOL) 判定阈值。

本题充电过程采用两阶段快充：以倍率 C_1 从 0% SOC 充至切换点 Q_1 ，再以倍率 C_2 从 Q_1 充至 80% SOC，随后以 1C 恒流–恒压 (CC-CV) 补至满电，至电流降至 $C/50$ 结束。各电池放电条件一致，主要差异在充电策略。数据来自 MIT–Stanford 公开循环老化实验 [1, 4]，包含 49 块额定容量 1.1 Ah 的 A123 18650 磷酸铁锂电池。

需要完成的研究任务为：

- 问题 1.** 整理数据，提取策略、充电时间、SOH 变化与循环寿命信息，分析寿命分布并解释典型长/短寿命策略差异。
- 问题 2.** 分析策略参数 C_1, Q_1, C_2 对循环寿命的影响及其统计显著性与可能机制。
- 问题 3.** 基于截至第 150 次循环的数据预测第 151–200 次循环 SOH，并外推 EOL 循环寿命。
- 问题 4.** 兼顾充电时间与寿命衰减，在实验策略及其合理邻域内优化快充策略。

2 问题分析与总体思路

问题一属于探索性数据分析：在尚未达到真实 EOL 的观测窗口内，需要定义可比较的寿命代理指标，并完成清洗、可视化与典型策略提取。问题二强调参数非正交，不宜按完全析因实验解释因果关系。问题三是早期循环预测与外推。问题四是充电时间与衰减率的多目标决策。四问递进关系为：数据整理 → 影响分析 → 寿命预测 → 策略优化。

待完成 / 框架占位

问题二至问题四的模型推导、数值表与最终推荐策略尚未写入仓库，下文相应小节仅给出可直接填充的结构、符号与拟采用方法，待后续结果补入后删除本框并替换为正式论述。

3 符号说明与基本假设

3.1 主要符号

表 1 主要符号说明

符号	含义
SOH	电池健康状态，见式(1)
C_1, C_2	第一、二阶段充电倍率 (C-rate)
Q_1	阶段切换 SOC (%)
N	循环次数
N_{EOL}	SOH = 80% 对应的循环寿命
SOH_{200}	第 200 次循环的 SOH (寿命代理)
T_{charge}	平均充电时间 (min)

3.2 基本假设

1. 各电池放电协议一致，寿命差异主要由充电策略及电池个体差异引起。
2. 在 200 次循环窗口内未达到 SOH = 80%，故问题一以 SOH_{200} 作为寿命代理指标，并在文中明确其与真实循环寿命的区别。
3. 问题一按充电配方将原始策略字符串统一为 7 类（去掉 NEWSTRUCTURE 后缀并合并同名记录）；合并组若组内离散明显，不作为典型策略。
4. IQR 筛出的离群点视为统计异常而非已证实的传感器故障，清洗仅用于本题指定分析。
5. 问题四若扩展连续参数，取值限制在已有实验数据的凸包或合理邻域内，避免无约束外推。

4 问题一：数据整理与快充策略对寿命影响的初步分析

4.1 数据来源与结构

本题使用赛题提供的锂离子电池循环老化数据 [1, 4] (MIT/Stanford 公开数据集)。battery_summary.csv 记录 49 块电池的策略参数 C_1, Q_1, C_2 、初始容量、平均充电时间、平均内阻、平均温度及问题三测试标记 prediction_test；cycle_train.csv 记录逐循环的容量、SOH、平滑 SOH、充电时间、内阻与平均温度，共 9350 行。非测试电池记录至第 200 次循环，9 块测试电池仅至第 150 次循环。输入质量检查表明：battery_id+cycle 无重复；循环表无缺失；汇总表有 3 个缺失单元格，其中部分电池 C_1 为结构性缺失，予以保留且不参与循环数据清洗。最大循环数分布为 {150 : 9, 200 : 40}。

4.2 寿命指标定义

题设循环寿命为 SOH 降至 80% 时的循环次数 N_{EOL} 。本数据集在 200 次循环内 SOH_{200} 范围为 0.9497 ~ 1.0002，均值 0.9914，尚未覆盖真实寿命终止。因此问题一采用

$$\text{LifeProxy}_i = \text{SOH}_i(N = 200) \quad (2)$$

作为观测窗口内的使用寿命代理：该值越高，表示前 200 圈健康保持越好。直方图与箱线图参考线取样本上、下四分位数，用于刻画分布位置，而不再直接等同于“寿命终止阈值”。若需与问题二、三衔接，可进一步计算衰减率

$$\text{FadeRate}_i = \frac{\text{SOH}_i(1) - \text{SOH}_i(N)}{N - 1}. \quad (3)$$

4.3 IQR 清洗

对每块电池、每个测量字段（capacity、SOH、SOH_smooth、chargetime、IR、Tavg）分别计算下、上四分位数 Q_1, Q_3 及四分位距 $\text{IQR} = Q_3 - Q_1$ 。将

$$x \notin [Q_1 - 1.5 \text{IQR}, Q_3 + 1.5 \text{IQR}]$$

的观测标为异常，并用同一电池、同一字段中距离最近的前后正常值均值替换；若仅有单侧邻域则用该侧值。共替换 559 个单元格，分字段数量为：capacity 36、SOH 36、SOH_smooth 17、IR 142、Tavg 142、chargetime 186。原始 data/ 文件未改写，清洗结果保存为 cycle_train_cleaned.csv。

4.4 快充策略统一命名

原始记录中共出现 9 种策略字符串。第三次迭代按充电配方将同名或仅差后缀的策略合并，并去掉 _NEWSTRUCTURE 后缀，得到 7 种统一策略，映射关系见表 2。验收确认：原始 9 种减为 7 种，旧名称无残留，49 块电池的 charge_policy.csv 均采用统一名称。

表 2 原始策略名称到统一策略名称的映射

原始名称	统一名称
3_6C-80PER_3_6C、80PER_3_6C	3_6C-80PER_3_6C
4_8C_80PER_4_8C、4_8C_80PER_4_8C_NEWSTRUCTURE	4_8C_80PER_4_8C
5C_67PER_4C_NEWSTRUCTURE	5C_67PER_4C
5_3C_54PER_4C_NEWSTRUCTURE	5_3C_54PER_4C
5_6C_19PER_4_6C_NEWSTRUCTURE	5_6C_19PER_4_6C
5_6C_36PER_4_3C_NEWSTRUCTURE	5_6C_36PER_4_3C
3_7C_31PER_5_9C_NEWSTRUCTURE	3_7C_31PER_5_9C

统一后的 7 种策略对应两阶段参数见表 3。其中 3_6C-80PER_3_6C 组部分电池原

始 C_1 缺失，表中记为“3.6/缺失”。寿命统计分析仅纳入最大循环数恰为 200 的 40 块电池，排除 9 块问题三测试电池。

表 3 统一策略的充电参数及过程量（200 圈样本）

统一策略	C_1	Q_1	C_2	n	$\overline{IR}$	$\overline{T}/^{\circ}C$	$\overline{T}_{ch}/min$
3_6C-80PER_3_6C	3.6/缺失	80	3.6	4	0.0170	31.84	13.382
4_8C_80PER_4_8C	4.8	80	4.8	7	0.0159	32.43	10.760
5C_67PER_4C	5.0	67	4.0	6	0.0155	34.04	10.043
5_3C_54PER_4C	5.3	54	4.0	7	0.0156	34.36	10.043
5_6C_19PER_4_6C	5.6	19	4.6	7	0.0152	35.41	10.043
5_6C_36PER_4_3C	5.6	36	4.3	7	0.0157	35.21	10.043
3_7C_31PER_5_9C	3.7	31	5.9	2	0.0139	35.68	10.134

注： n 为满 200 圈电池数；充电时间、内阻与温度为各电池 1-200 圈均值后再按策略平均。

4.5 SOH 变化曲线

全部曲线均采用清洗后的 SOH_smooth。图 1 给出 1 号电池示例；49 块电池的单体曲线见支撑材料 A/问题一/SOH/SOH_N.png。图 2 将全部电池叠绘于同一坐标系，颜色由蓝到红随 battery_id 递增。可见多数电池在 200 圈内衰减缓慢，少数曲线明显下探，后文箱线图表明这些快速衰减样本主要集中于 3_7C_31PER_5_9C 以及合并后离散较大的两组。

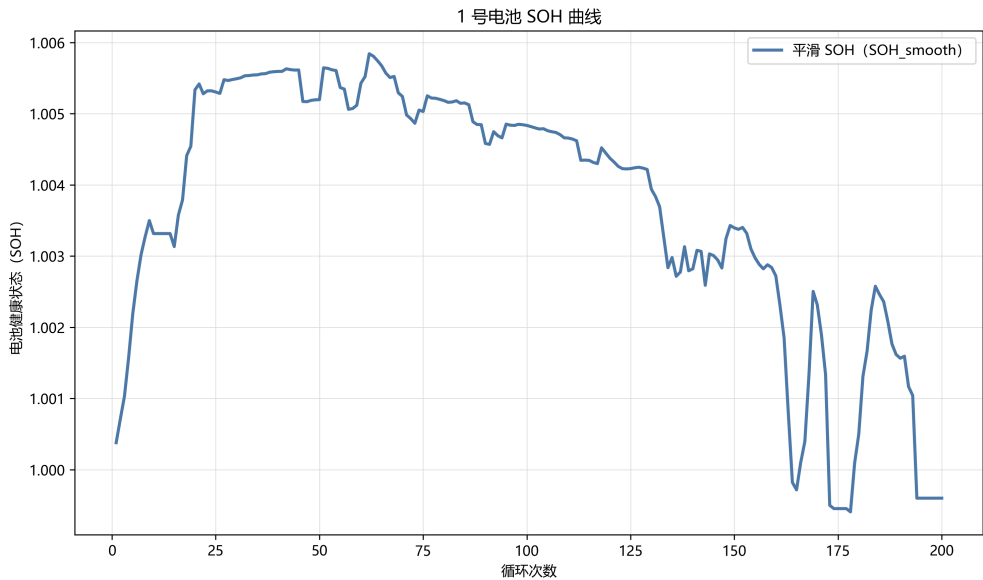


图 1 1 号电池平滑 SOH 曲线

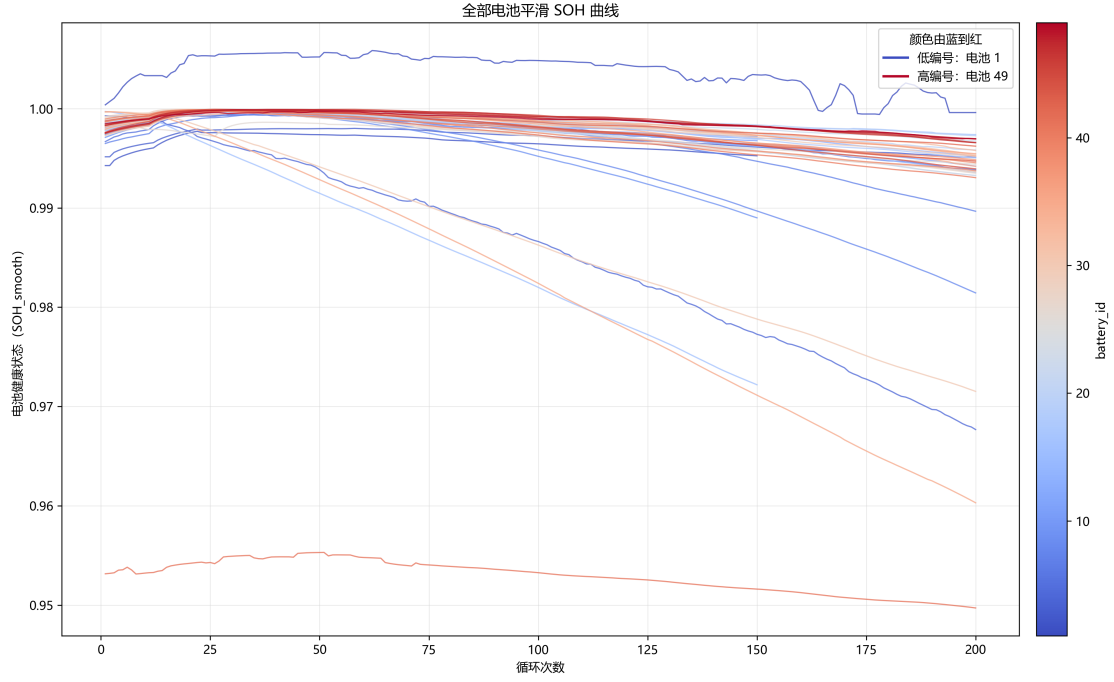


图 2 全部 49 块电池的平滑 SOH 曲线（颜色由蓝到红对应电池编号增大）

4.6 寿命分布统计分析

对 40 块满 200 圈电池的 SOH_{200} 绘制直方图，见图 3。样本上、下四分位

$$Q_3 = 0.996103, \quad Q_1 = 0.993433$$

分别以红色、蓝色虚线标出，用作分布位置参考。据此可将单体分为高于 Q_3 的 10 块、介于 Q_1 与 Q_3 的 20 块、低于 Q_1 的 10 块。需要强调：该划分针对的是**电池个体**，且全样本 SOH_{200} 高度集中在 0.99 附近，多数 $C_1 > C_2$ 策略的电池落在中间箱体。因此不能把“策略中位数是否越过 Q_3 ”直接等同于“典型长寿命策略”；后文改按组内集中程度与 C_1, C_2 形态提取典型策略。

SOH_{200} 与平均充电时间的散点图见图 4。四类 $C_1 > C_2$ 的分段快充策略充电时间均集中在约 10.04 min，SOH 也较高；3_7C_31PER_5_9C 充电时间相近（约 10.13 min）但 SOH 明显更低，说明在相近补能时间下，策略形态仍可造成寿命差异。3_6C-80PER_3_6C 充电时间最长（约 13.38 min），但 SOH 并不一律最高。4_8C_80PER_4_8C 的充电时间分裂为约 10.06 min（电池 7、8）与约 11.04 min（电池 17、20、41、46、49）两簇，后者中位数虽高，却包含 $SOH_{200} \approx 0.950$ 的显著偏低个体（电池 41）。

不同统一策略下 SOH_{200} 的箱线图见图 5，策略汇总见表 4。

4.7 典型长寿命与短寿命策略

箱线图显示，3_6C-80PER_3_6C 与 4_8C_80PER_4_8C 的 SOH_{200} 在组内大幅波动，不宜作为“典型”代表。前者由 80PER_3_6C 与 3_6C-80PER_3_6C 合并，部分电池 C_1

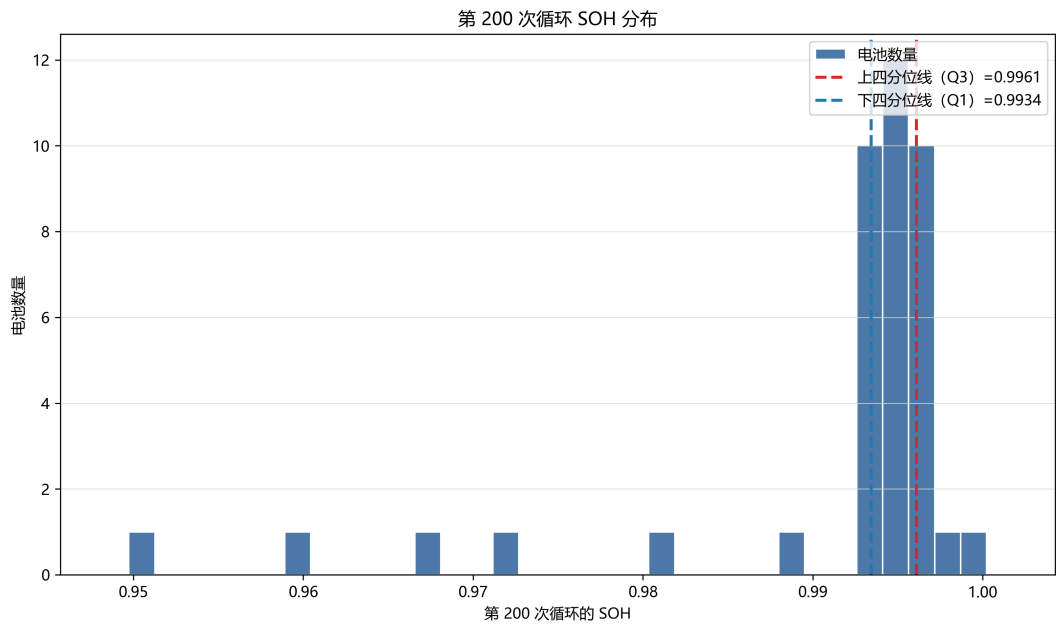


图 3 第 200 次循环 SOH 分布及上、下四分位参考线

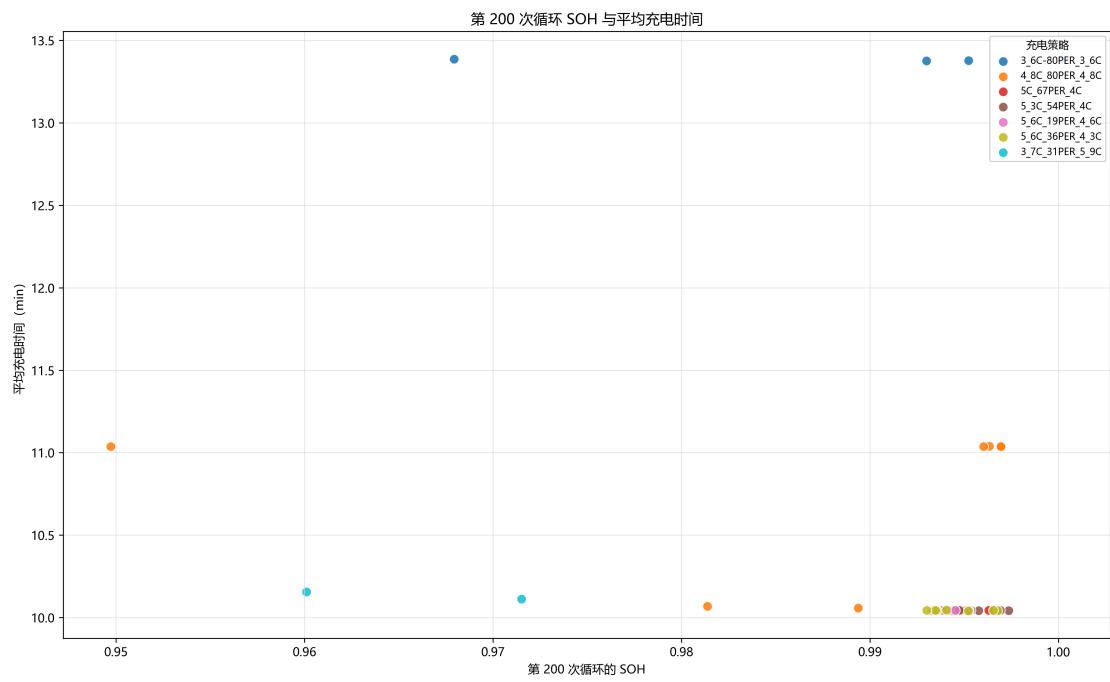


图 4 第 200 次循环 SOH 与平均充电时间散点图（按统一策略着色）

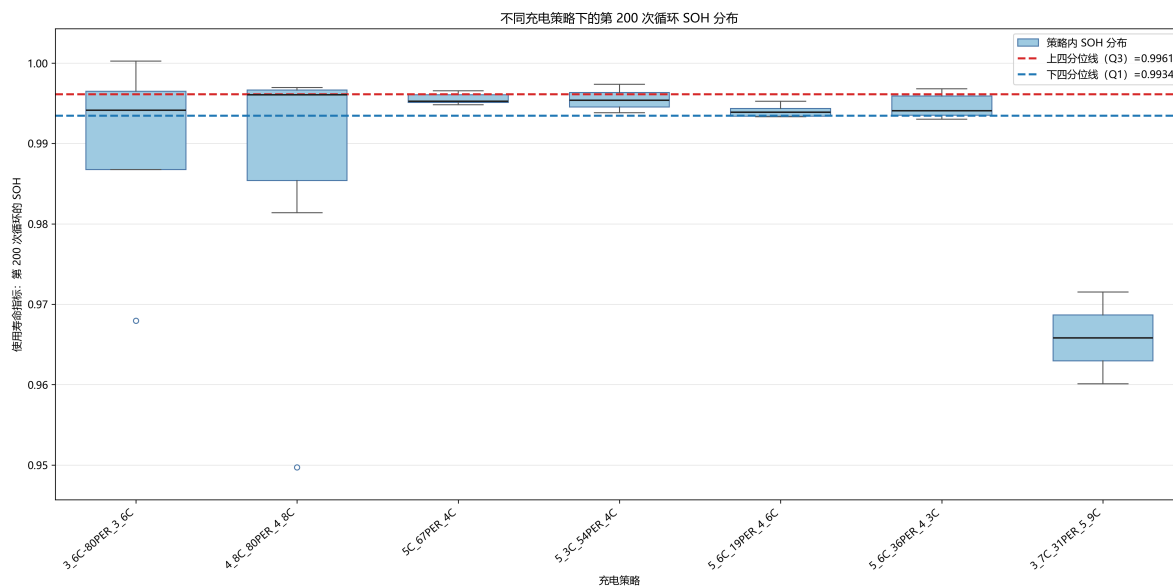


图 5 不同统一充电策略下第 200 次循环 SOH 箱线图

表 4 统一策略第 200 次循环 SOH（均值/中位数）与充电时间汇总

统一策略	电池记号	n	SOH 平均	中位数	SOH 范围	充电时间/min
5C_67PER_4C	18,29,31,37,42,47	6	0.9955	0.9952	0.9948–0.9966	10.042–10.044
5_3C_54PER_4C	19,26,32,36,38,43,48	7	0.9955	0.9954	0.9938–0.9974	10.042–10.044
5_6C_36PER_4_3C	13,15,22,28,34,40,45	7	0.9947	0.9941	0.9930–0.9968	10.041–10.045
5_6C_19PER_4_6C	12,21,23,27,33,39,44	7	0.9940	0.9939	0.9933–0.9953	10.042–10.044
3_6C_80PER_3_6C	1,3,4,6	4	0.9891	0.9941	0.9679–1.0002	13.376–13.388
4_8C_80PER_4_8C	7,8,17,20,41,46,49	7	0.9867	0.9960	0.9497–0.9970	10.058–11.038
3_7C_31PER_5_9C	30,35	2	0.9658	0.9658	0.9601–0.9715	10.111–10.156

注：按 SOH 平均值降序。若改按中位数，则 4_8C_80PER_4_8C 最高（流水线据此标记“寿命最长”），但其均值被低尾个体拉低。测试电池未纳入。完整参数见 policy.csv。

缺失，既含全样本最高个体（电池 1， $\text{SOH}_{200} \approx 1.000$ ），也含明显偏低个体（电池 4， ≈ 0.968 ）；后者合并带/不带 NEWSTRUCTURE 的记录，充电时间分为约 10.06 min 与 11.04 min 两簇。流水线若只按策略 SOH 中位数会把 4_8C_80PER_4_8C 标为寿命最长，但这恰好说明：高中位数可以被少数高健康样本抬起，而不能代表该配方的稳健水平。因此典型策略改从组内更集中的分段快充中提取，与 A/问题一/总结.md 的判定一致。

典型长寿命策略（组内 SOH 高且离散小，充电时间约 10.04 min）：

- 5C_67PER_4C: $C_1 = 5.0 > C_2 = 4.0$, $Q_1 = 67\%$, SOH 平均 0.9955;
- 5_3C_54PER_4C: $C_1 = 5.3 > C_2 = 4.0$, $Q_1 = 54\%$, SOH 平均 0.9955;
- 5_6C_36PER_4_3C: $C_1 = 5.6 > C_2 = 4.3$, $Q_1 = 36\%$, SOH 平均 0.9947;
- 5_6C_19PER_4_6C: $C_1 = 5.6 > C_2 = 4.6$, $Q_1 = 19\%$, SOH 平均 0.9940。

典型短寿命策略：3_7C_31PER_5_9C, $C_1 = 3.7 < C_2 = 5.9$, $Q_1 = 31\%$, SOH 平均仅 0.9658，两块电池均明显低于下四分位线。其平均充电时间约 10.13 min，与长寿命组几乎同量级，因此衰减差异主要来自策略形态而非“充得更快”。

4.8 长、短寿命策略差异的解释

结合两阶段快充的物理图像 [2,3,5,6]，可将问题一的关键差异概括如下。

1. **电流分配**： $C_1 > C_2$ 优于 $C_1 < C_2$ 。典型长寿命策略均满足前段倍率高于后段，即在低 SOC 区间采用较大电流、在中高 SOC 区间回落到较低电流。短寿命策略则相反：后段 $C_2 = 5.9C$ 为全样本最高。高 SOC 时电解液中锂离子浓度梯度更大、石墨负极嵌锂位点更拥挤，过电位升高后更易诱发锂沉积与不可逆容量损失；同时极化与产热叠加，加速 SEI 生长 [2,3]。LFP 体系虽相对耐过充，但仍可通过活性锂损失（LLI）表现为早期 SOH 下降 [5]。多段恒流对照亦表明，把大电流尽量安排在低 SOC 段更有利于寿命 [6]。
2. **恒流高 SOC 充电**：提高 $C_1 = C_2$ 时均值与下尾变差，但中位数并不同步。3.6C(80%)–3.6C 与 4.8C(80%)–4.8C 两组均为 $Q_1 = 80\%$ 、两阶段倍率相同，相当于把高倍率维持到较高 SOC。4.8C 组均值（0.9867）略低于 3.6C 组（0.9891），且下尾更长（最低约 0.950）；但其中位数反而更高（0.9960 对 0.9941）。因此“提高恒流倍率则 SOH 下降”只在均值与最差个体意义上可见，不能由中位数单独下结论，这两组也不宜当作典型对照。
3. **相近充电时间下仍可拉开寿命**。长寿命四策略与短寿命策略的平均充电时间均约 10.0 ~ 10.2 min，说明“时间短”既不是短寿的充分条件，也不是长寿的必要条件。问题四的优化应同时约束时间与衰减，而不能只最小化充电时间。
4. **提高总体电流时，降低 Q_1 有助于保护寿命**。在 $C_1 > C_2$ 的四组中，四策略通过调整 Q_1 与 C_1, C_2 使平均充电时间几乎同为 10.04 min，但 SOH 仍可在窄区间内变化： Q_1 从 67% 降至 19% 时，高倍率持续时间缩短、后段较低电流覆盖更宽的中

高 SOC 区间。四组 SOH 均值均明显高于短寿命策略。更精细的 Q_1 效应与显著性检验留待问题二。

5. 温度、内阻与初始容量不能单独解释长短寿差异。短寿命组平均温度最高（约 35.7°C ），但长寿命的 5.6C 两组温度同样约 $35.2 \sim 35.4^{\circ}\text{C}$ ，SOH 却明显更高；短寿命组平均内阻反而是最低（约 0.0139Ω ），初始容量各组均在 $1.05 \sim 1.07\text{Ah}$ 。因此在本观测窗口内，策略形态（何时施加大电流）比这三项过程量更能区分典型长、短寿命。
6. 合并组的高离散度提示标签/批次影响。统一命名便于后续建模，但不宜把合并组的中位数直接当作该配方的稳健寿命水平；问题二应对这两组合并前后做稳健性对照。

上述结论仅基于第 200 次循环的健康保持程度，样本中若干策略仅 2-4 块满 200 圈电池，相关关系不能直接等同于因果效应。问题二将在统一策略与原始参数两个层面检验 C_1, Q_1, C_2 的统计影响。

5 问题二：充电策略及其参数对寿命衰减的影响

待完成 / 框架占位

本节为建模框架。仓库中尚无问题二的专用脚本与统计表。补全后应删除本框，填入检验统计量、回归系数与诊断图。

5.1 问题分析

问题一已表明策略间 SOH_{200} 存在可见差异，但 C_1, Q_1, C_2 并非完全独立变化，策略数仅 9、部分组 $n = 2$ 。因此本节重点是：组间差异是否具有统计显著性；三参数与寿命的相关/回归关系；不同 SOC 区间高倍率是否对应不同衰减特征；以及机制解释的局限性 [7-9]。

5.2 数据与可比性规则（拟采用）

1. 主分析：满 200 圈样本；因变量优先用 SOH_{200} 或 FadeRate。
2. 公平对比：以统一后的 7 类策略为主分析；对合并后离散很大的 3_6C-80PER_3_6C、4_8C_80PER_4_8C 可作稳健性检验（拆回原始标签）。
3. 若纳入测试电池，统一截断至第 150 圈再比较，避免窗口不一致。

5.3 模型一：策略间差异的显著性检验

将策略视为分组因子。若各组近似正态且方差齐，采用单因素 ANOVA；否则采用 Kruskal–Wallis 检验，事后比较用 Tukey HSD 或 Dunn 检验，并报告效应量 η^2 [10]。

待填结果： H 或 F 统计量、 p 值、两两比较中显著的策略对。

5.4 模型二：参数与寿命的回归 / 相关分析

考察

$$\text{LifeProxy}_i = \beta_0 + \beta_1 C_{1,i} + \beta_2 Q_{1,i} + \beta_3 C_{2,i} + \beta_4 C_{2,i} \cdot \mathbf{1}\{Q_{1,i} < Q^*\} + \varepsilon_i. \quad (4)$$

因参数共线，可同时报告 Spearman 相关、方差膨胀因子（VIF），必要时采用混合效应（策略随机截距）[11]。交互项用于刻画“高倍率发生在低/高 SOC 区间”的差异。

待填结果：系数表、标准化系数或变量重要性、残差诊断。

5.5 不同 SOC 区间高倍率的衰减特征

将充电过程拆为低 SOC 段 $[0, Q_1]$ 与高 SOC 段 $[Q_1, 80\%]$ ，比较 C_1 主导段与 C_2 主导段对 FadeRate 的边际贡献。预期：在数据允许时， C_2 对寿命的负向作用强于同等幅度的 C_1 [2,6]。

5.6 机制讨论与局限

拟从 SEI 生长、锂沉积、极化与产热等 LFP 相关路径讨论 [5,12,13]，并明确：策略非正交、早期衰减幅度小、cell-to-cell 变异与批次标签会限制因果识别。

6 问题三：基于早期循环的电池寿命预测

待完成 / 框架占位

本节为建模框架。9 块测试电池已在 `battery_summary.csv` 中由 `prediction_test=1` 标记，循环数据截至第 150 圈。预测表、误差指标与 EOL 外推结果待补入。

6.1 问题分析

实际中往往只能获得前期循环，无法等待完整老化。本题要求：用截至第 150 次循环的数据预测第 151–200 次循环 SOH，并进一步估计 $\text{SOH} = 80\%$ 的循环寿命；同时比较不同早期窗口长度，讨论策略信息与早期衰减特征的作用 [1,9,14–16]。

6.2 特征提取（拟采用）

表 5 问题三拟提取的特征类别

类别	示例
策略参数	C_1, Q_1, C_2 及策略编码
衰减趋势	前 k 圈 SOH 斜率、曲率、幂律/指数拟合参数
过程量	充电时间、IR、温度的均值与变化率
分段差	$\Delta\text{SOH}_{1 \rightarrow 50}$ 、 $\Delta\text{SOH}_{50 \rightarrow 100}$ 、 $\Delta\text{SOH}_{100 \rightarrow 150}$

优先使用 SOH_smooth。若后续引入放电电压曲线类特征 $\Delta Q(V)$ ，需回源完整公开数据 [4, 14]。

6.3 预测模型（拟采用）

经验衰减模型（可解释）：

$$\text{SOH}(n) = 1 - a n^b \quad \text{或} \quad \text{SOH}(n) = a e^{-bn} + c, \quad (5)$$

用 1–150 圈拟合，外推 151–200 及 N_{EOL} 。

统计 / 机器学习模型：以早期统计特征预测后续 SOH 或衰减率，候选包括正则化线性模型、随机森林 / XGBoost、高斯过程；序列模型可试 LSTM [15]。验证建议：在 40 块非测试电池上做“截断 150 圈 → 预测 151–200”的交叉验证，再对 9 块测试电池给出最终预报，避免同策略信息泄漏。

6.4 评价指标与窗口比较

对 151–200 圈报告 RMSE、MAE 与 R^2 ；对 N_{EOL} 报告外推圈数。比较输入窗口 $k \in \{50, 100, 150\}$ 的精度差异。消融：有/无 (C_1, Q_1, C_2) 。

待填：9 块测试电池的 SOH 预测曲线、误差表、EOL 点估计。

7 问题四：兼顾充电时间与寿命衰减的策略优化

待完成 / 框架占位

本节为建模框架。优化应优先在问题一统一后的 7 种实验策略内比较；若连续优化，参数须限制在实验数据合理邻域并说明依据 [17–20]。

7.1 决策变量与目标

决策变量 $x = (C_1, Q_1, C_2)$ 。目标为在较短充电时间下降低衰减速率或延长寿命：

$$\min_x F(x) = \begin{bmatrix} T_{\text{charge}}(x) \\ \text{DecayRate}(x) \end{bmatrix} \quad \text{或} \quad \min_x w_1 T_{\text{charge}}(x) + w_2 \text{DecayRate}(x). \quad (6)$$

7.2 充电时间模型（拟采用）

分段恒流近似

$$T_{\text{charge}} \approx \frac{Q_1}{C_1} + \frac{0.8 - Q_1}{C_2} + T_{\text{CV}}, \quad (7)$$

并用 `mean_chargetime` 回归校准。问题一散点图已显示充电时间与策略分组相关，可用于检验该近似。

7.3 寿命衰减模型

沿用问题二回归或问题三预测模型，得到 $\text{DecayRate}(x)$ 或 $\text{SOH}_{200}(x)$ 。在已有策略上先做离散 Pareto 比较，再考虑邻域连续搜索。

7.4 约束与邻域

建议参数盒约束取自样本极值（以最终校核为准）： $C_1 \in [3.6, 5.6]$ ， $Q_1 \in [19, 80]$ ， $C_2 \in [3.6, 5.9]$ 。超出实验凸包的组合须标明外推风险。

7.5 推荐策略与对比（待填）

给出推荐 (C_1^*, Q_1^*, C_2^*) ，并与问题一的典型长寿命策略（如 5C_67PER_4C、5_3C_54PER_4C）及短寿命策略 3_7C_31PER_5_9C 对比充电时间与寿命代理。说明适用范围（LFP、两阶段快充、早期循环窗口）与不可外推情形。

8 结论与展望

已完成结果（仓库现有成果）

问题一已形成的结论：

- 循环数据经 IQR 清洗后可用于策略对比；原始 9 种策略统一为 7 类。
- 在满 200 圈的 40 块电池中，第 200 次循环 SOH 均值为 0.9914，尚未达到 80% EOL。
- 按中位数会把 4.8C 合并组标为最长，但其组内离散大、充电时间分两簇，故不以该组为典型；典型长寿命满足 $C_1 > C_2$ （如 5C(67%) – 4C 等），充电时间约 10.04 min；典型短寿命为 3.7C(31%) – 5.9C ($C_1 < C_2$)。
- 相近充电时间下，高 SOC 区间高倍率会明显加快衰减； $Q_1 = 80\%$ 的恒流合并组离散大，提高倍率时均值与下尾变差但中位数并不同步，故不作典型对照；提高电流时降低 Q_1 有助于保持 SOH。
- 温度、内阻与初始容量不能单独解释长短寿差异。

待完成 / 框架占位

补充问题二显著性与参数效应结论、问题三预测精度与 EOL 估计、问题四推荐策略及与长短寿命策略的对比；讨论模型局限与可推广范围。

参考文献

参考文献

- [1] Kristen A. Severson, Peter M. Attia, Norman Jin, Nicholas Perkins, Benben Jiang, Zi Yang, Michael H. Chen, Muratahan Aykol, Patrick K. Herring, Dimitrios Fraggedakis, Martin Z. Bazant, Stephen J. Harris, William C. Chueh, and Richard D. Braatz. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5):383–391, 2019. 本题主数据集来源；问题 1-4 必引。
- [2] Yayuan Liu, Yangying Zhu, and Yi Cui. Challenges and opportunities towards fast-charging battery materials. *Nature Energy*, 4:540–550, 2019. 快充机理综述；问题 1 机制解释。
- [3] Shabbir Ahmed, Ira Bloom, Andrew N. Jansen, Tanvir R. Tanim, Eric J. Dufek, Ahmad Pesaran, et al. Enabling fast charging – A battery technology gap assessment. *Journal of Power Sources*, 367:250–262, 2017. 快充技术缺口评估；问题 1/2 背景。

- [4] MIT–Stanford–TRI battery dataset. <https://data.matr.io/1/>. Severson 2019 配套公开数据；本题主数据.
- [5] Matthieu Dubarry and Bor Yann Liaw. Identify capacity fading mechanism in a commercial LiFePO₄ cell. *Journal of Power Sources*, 194:541–549, 2009. LFP 衰减机理；问题 1/2.
- [6] Y. Ma et al. An aging-optimized state-of-charge-controlled multi-stage constant current (MCC) fast charging algorithm for commercial Li-ion battery based on three-electrode measurements. *Batteries*, 10(8):267, 2024. 多段恒流快充；问题 2/4.
- [7] P. L. Ruiz et al. Physics-based and data-driven modeling of degradation mechanisms for lithium-ion batteries: A review. *Batteries*, 2023. TU Delft review; Problem 2.
- [8] H. Wang et al. High accuracy and applicability battery aging models for electric vehicle applications. *Journal of The Electrochemical Society*, 2024. 半经验老化模型；问题 2.
- [9] Alexis Geslin et al. Commentary on charging features for battery lifetime prediction. *Joule*, 2023. Charging-condition features and lifetime prediction; Problem 2/3.
- [10] Michael H. Kutner, Christopher J. Nachtsheim, John Neter, and William Li. *Applied Linear Statistical Models*. McGraw-Hill, 5 edition, 2005. 问题 2 ANOVA/回归.
- [11] José Pinheiro and Douglas Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, 2000. 问题 2 混合效应模型.
- [12] Matthieu Dubarry, Cyril Truchot, and Bor Yann Liaw. Synthesize battery degradation modes via a diagnostic and prognostic model. *Journal of Power Sources*, 219:204–216, 2012. 退化模式诊断框架；问题 1/2.
- [13] W. Li et al. Review of state-of-the-art degradation models for lithium-ion batteries. *Entropy*, 28(6):669, 2026. 退化模型新综述；问题 2.
- [14] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, et al. Statistical learning for accurate and interpretable battery lifetime prediction. *Journal of The Electrochemical Society*, 2021. 问题 3 特征工程与可解释预测.
- [15] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. *IEEE Transactions on Vehicular Technology*, 67(7):5695–5705, 2018. LSTM-RNN and Monte Carlo probabilistic RUL prediction.

- [16] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Lithium-ion battery remaining useful life prediction with box-cox transformation and monte carlo simulation. *IEEE Transactions on Industrial Electronics*, 66(2):1585–1597, 2019. Box-Cox transformation and Monte Carlo simulation for online RUL prediction.
- [17] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, Todor M. Markov, Yang Liao, Michael H. Chen, Bryan Cheong, Nicholas Perkins, Zi Yang, Patrick K. Herring, Muratahan Aykol, Stephen J. Harris, Richard D. Braatz, Stefano Ermon, and William C. Chueh. Closed-loop optimization of fast-charging protocols for batteries with machine learning. *Nature*, 578:397–402, 2020. 问题 3 早期预测 + 问题 4 贝叶斯优化快充协议.
- [18] Gregory L. Plett and M. Scott Trimboli. *Battery Management Systems, Volume III: Physics-Based Methods*. Artech House, 2022. 物理模型与快充优化；问题 4.
- [19] Peter I. Frazier. A tutorial on Bayesian optimization, 2018. 问题 4 贝叶斯优化方法.
- [20] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001. 问题 4 多目标优化 / NSGA-II.

A 支撑材料文件列表

按赛制，支撑材料单独打包为压缩文件（RAR/ZIP，不超过 20 MB），本附录列出其内容。赛题原始数据 `data/battery_summary.csv`、`data/cycle_train.csv` 不作为“自主查阅数据”重复提交，但建模程序依赖它们运行。

表 6 支撑材料拟包含的文件

路径	说明
<code>src/A/task1_pipeline.py</code>	问题一完整可运行流水线
<code>src/A/SOH_plot.py</code>	SOH 曲线绘制模块
<code>A/问题一/AGENTS.md</code>	问题一任务说明（提示词）
<code>A/问题一/log_1.md-log_3.md</code>	运行日志
<code>A/问题一/总结.md</code>	问题一结论摘要
<code>A/问题一/output/A/cycle_train_cleaned.csv</code>	清洗后循环数据
<code>A/问题一/output/A/charge_policy.csv</code>	统一后的电池-策略表
<code>A/问题一/output/A/battery_SOH_charge.200</code>	200 圈 SOH 与充电时间
<code>A/问题一/output/A/policy.csv</code>	7 类策略汇总
<code>A/问题一/output/A/doc/1.md</code>	策略统计报告
<code>A/问题一/output/A/acceptance_3.md</code>	第三次迭代验收
<code>A/问题一/output/A/*.png</code>	问题一图件
<code>A/问题一/SOH/</code>	49 张单体曲线与总图 SOH_ALL
<code>references/</code>	参考文献导读与 BibTeX
<code>AI 使用记录/</code>	AI 使用详情底稿
<code>paper/</code>	本 LaTeX 工程

问题二至问题四的脚本、中间表与预测结果待生成后追加本表。若最终提交时上述文件均已纳入压缩包，则与论文结论对应；源程序运行方式见附录 B。

B 建模源程序

本节给出问题一全部可运行源代码。运行环境：Python 3，依赖 numpy、pandas、matplotlib。在项目根目录执行：

```
1 python src/A/task1_pipeline.py
```

可选参数 `--iqr-factor`（默认 1.5）。单独绘制指定电池 SOH 曲线：

```
1 python src/A/SOH_plot.py --battery-id 1
```

待完成 / 框架占位

问题二–四源程序完成后，以同样方式用 `\lstinputlisting` 追加完整代码，确保运行结果与正文图表一致。

B.1 SOH_plot.py

Listing 1 SOH 曲线绘制脚本

```
1  """绘制指定电池的 SOH 曲线。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from pathlib import Path
7
8  import matplotlib
9
10 matplotlib.use("Agg")
11
12 import matplotlib.font_manager as font_manager
13 import matplotlib.pyplot as plt
14 import numpy as np
15 import pandas as pd
16
17
18 PROJECT_ROOT = Path(__file__).resolve().parents[2]
19
20
21 def configure_chinese_font() -> str:
```

```

22     """选择系统中可用的中文字体，并返回字体名称。"""
23     installed = {font.name for font in font_manager.fontManager.ttflist}
24     candidates = [
25         "Microsoft YaHei",
26         "SimHei",
27         "Noto Sans CJK SC",
28         "Source Han Sans SC",
29         "Arial Unicode MS",
30     ]
31     selected = next((name for name in candidates if name in installed), "DejaVu
        Sans")
32     plt.rcParams["font.sans-serif"] = [selected, "DejaVu Sans"]
33     plt.rcParams["axes.unicode_minus"] = False
34     return selected
35
36
37 def plot_battery_soh(
38     cycles: pd.DataFrame,
39     battery_id: int,
40     output_path: Path,
41 ) -> None:
42     """绘制指定电池的平滑 SOH 曲线。"""
43     required = {"battery_id", "cycle", "SOH_smooth"}
44     missing = required.difference(cycles.columns)
45     if missing:
46         raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
47
48     battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("cycle"
        )
49     if battery.empty:
50         raise ValueError(f"未找到 battery_id={battery_id} 的数据")
51
52     configure_chinese_font()
53     fig, ax = plt.subplots(figsize=(10, 6))
54     ax.plot(
55         battery["cycle"],
56         battery["SOH_smooth"],
57         color="#4C78A8",
58         linewidth=2.2,
59         label="平滑 SOH (SOH_smooth) ",
60     )
61     ax.set_title(f"{battery_id} 号电池 SOH 曲线")
62     ax.set_xlabel("循环次数")
63     ax.set_ylabel("电池健康状态 (SOH) ")

```

```

64 ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
65 ax.legend(loc="upper right")
66 fig.tight_layout()
67 output_path.parent.mkdir(parents=True, exist_ok=True)
68 fig.savefig(output_path, dpi=300, bbox_inches="tight")
69 plt.close(fig)
70
71
72 def plot_all_battery_soh(cycles: pd.DataFrame, output_path: Path) -> None:
73     """用蓝到红渐变绘制全部电池的平滑 SOH 曲线。"""
74     required = {"battery_id", "cycle", "SOH_smooth"}
75     missing = required.difference(cycles.columns)
76     if missing:
77         raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
78
79     battery_ids = sorted(int(value) for value in cycles["battery_id"].unique())
80     if not battery_ids:
81         raise ValueError("循环数据中没有电池记录")
82
83     configure_chinese_font()
84     fig, ax = plt.subplots(figsize=(14, 8))
85     colormap = matplotlib.colormaps.get_cmap("coolwarm")
86     colors = colormap(np.linspace(0, 1, len(battery_ids)))
87     for color, battery_id in zip(colors, battery_ids):
88         battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("
            cycle")
89         ax.plot(
90             battery["cycle"],
91             battery["SOH_smooth"],
92             color=color,
93             linewidth=1.05,
94             alpha=0.78,
95         )
96
97     ax.set_title("全部电池平滑 SOH 曲线")
98     ax.set_xlabel("循环次数")
99     ax.set_ylabel("电池健康状态 (SOH_smooth)")
100     ax.grid(True, color="#D9D9D9", linewidth=0.6, alpha=0.6)
101     ax.legend(
102         handles=[
103             plt.Line2D([0], [0], color=colors[0], linewidth=2, label=f"低编号: 电
                池 {battery_ids[0]}"),
104             plt.Line2D([0], [0], color=colors[-1], linewidth=2, label=f"高编号: 电
                池 {battery_ids[-1]}"),

```

```

105     ],
106     loc="upper right",
107     title="颜色由蓝到红",
108 )
109 scalar_mappable = plt.cm.ScalarMappable(
110     cmap=colormap,
111     norm=plt.Normalize(vmin=battery_ids[0], vmax=battery_ids[-1]),
112 )
113 scalar_mappable.set_array([])
114 colorbar = fig.colorbar(scalar_mappable, ax=ax, pad=0.015)
115 colorbar.set_label("battery_id")
116 fig.tight_layout()
117 output_path.parent.mkdir(parents=True, exist_ok=True)
118 fig.savefig(output_path, dpi=300, bbox_inches="tight")
119 plt.close(fig)
120
121
122 def parse_args() -> argparse.Namespace:
123     parser = argparse.ArgumentParser(description="绘制指定电池的 SOH 曲线")
124     parser.add_argument("--battery-id", type=int, default=1, help="电池编号，默认
125         为 1")
126     parser.add_argument(
127         "--input",
128         type=Path,
129         default=PROJECT_ROOT / "output" / "A" / "cycle_train_cleaned.csv",
130         help="循环数据 CSV 路径",
131     )
132     parser.add_argument(
133         "--output",
134         type=Path,
135         default=PROJECT_ROOT / "output" / "A" / "SOH_1_example.png",
136         help="输出图像路径",
137     )
138     return parser.parse_args()
139
140 def main() -> None:
141     args = parse_args()
142     cycles = pd.read_csv(args.input)
143     plot_battery_soh(cycles, args.battery_id, args.output)
144     print(f"已生成: {args.output}")
145
146
147 if __name__ == "__main__":

```

```
148 main()
```

B.2 task1_pipeline.py

Listing 2 问题一分析流水线

```
1  """问题一：数据清洗、策略提取、200 次循环汇总与绘图。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from collections import Counter
7  from datetime import datetime
8  from pathlib import Path
9
10 import matplotlib
11
12 matplotlib.use("Agg")
13
14 import matplotlib.pyplot as plt
15 from matplotlib.patches import Patch
16 import numpy as np
17 import pandas as pd
18
19 from SOH_plot import configure_chinese_font, plot_all_battery_soh,
20     plot_battery_soh
21
22 PROJECT_ROOT = Path(__file__).resolve().parents[2]
23 MEASUREMENT_COLUMNS = ["capacity", "SOH", "SOH_smooth", "chargetime", "IR", "
24     Tavg"]
25
26 def normalize_policy_name(policy: str) -> str:
27     """按第三次迭代规则合并策略并删除 NEWSTRUCTURE 后缀。"""
28     normalized = str(policy).strip()
29     direct_mapping = {
30         "80PER_3_6C": "3_6C-80PER_3_6C",
31         "3_6C-80PER_3_6C": "3_6C-80PER_3_6C",
32         "4_8C_80PER_4_8C_NEWSTRUCTURE": "4_8C_80PER_4_8C",
33         "4_8C_80PER_4_8C": "4_8C_80PER_4_8C",
34     }
35     if normalized in direct_mapping:
36         return direct_mapping[normalized]
```

```

37     upper = normalized.upper()
38     if upper.endswith("_NEWSTRUCTURE"):
39         normalized = normalized[: -len("_NEWSTRUCTURE")]
40     return normalized
41
42
43 def normalize_policies(
44     summary: pd.DataFrame,
45     cycles: pd.DataFrame,
46 ) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, str]]:
47     """在处理副本中统一两个数据表的策略名称，原始数据保持不变。"""
48     normalized_summary = summary.copy()
49     normalized_cycles = cycles.copy()
50     original_names = sorted(str(value) for value in summary["policy"].unique())
51     mapping = {name: normalize_policy_name(name) for name in original_names}
52     normalized_summary["policy"] = normalized_summary["policy"].map(mapping)
53     normalized_cycles["policy"] = normalized_cycles["policy"].map(mapping)
54     if normalized_summary["policy"].isna().any() or normalized_cycles["policy"].
55         isna().any():
56         raise ValueError("策略统一命名过程中产生空值")
57     return normalized_summary, normalized_cycles, mapping
58
59 def validate_inputs(summary: pd.DataFrame, cycles: pd.DataFrame) -> None:
60     summary_required = {"battery_id", "policy", "initial_capacity", "
61         prediction_test"}
62     cycle_required = {
63         "battery_id",
64         "cycle",
65         "capacity",
66         "SOH",
67         "SOH_smooth",
68         "chargetime",
69         "IR",
70         "Tavg",
71         "policy",
72     }
73     summary_missing = summary_required.difference(summary.columns)
74     cycle_missing = cycle_required.difference(cycles.columns)
75     if summary_missing or cycle_missing:
76         raise ValueError(
77             f"输入字段不完整; battery_summary 缺少 {sorted(summary_missing)}, "
78             f"cycle_train 缺少 {sorted(cycle_missing)}"
79         )

```

```

79     if summary["battery_id"].duplicated().any():
80         raise ValueError("battery_summary.csv 中 battery_id 不唯一")
81     if cycles.duplicated(["battery_id", "cycle"]).any():
82         raise ValueError("cycle_train.csv 中存在重复的 battery_id + cycle")
83     if set(summary["battery_id"]) != set(cycles["battery_id"]):
84         raise ValueError("两个输入文件的 battery_id 集合不一致")
85
86     policy_check = cycles.merge(
87         summary[["battery_id", "policy"]],
88         on="battery_id",
89         suffixes=("_cycle", "_summary"),
90         validate="many_to_one",
91     )
92     if (policy_check["policy_cycle"] != policy_check["policy_summary"]).any():
93         raise ValueError("cycle_train.csv 与 battery_summary.csv 的策略字段不一致")
94
95
96     def iqr_clean_by_battery(
97         cycles: pd.DataFrame,
98         columns: list[str],
99         factor: float = 1.5,
100     ) -> tuple[pd.DataFrame, list[dict[str, float | int | str]]]:
101         """逐电池、逐字段进行 IQR 检测，并用最近前后正常值的均值替换。"""
102         cleaned = cycles.sort_values(["battery_id", "cycle"]).copy()
103         audit: list[dict[str, float | int | str]] = []
104
105         for battery_id, group in cycles.sort_values(["battery_id", "cycle"]).groupby(
106             "battery_id":
107         ):
108             for column in columns:
109                 values = group[column].to_numpy(dtype=float)
110                 finite_values = values[np.isfinite(values)]
111                 if finite_values.size == 0:
112                     continue
113
114                 q1, q3 = np.quantile(finite_values, [0.25, 0.75])
115                 iqr = q3 - q1
116                 if np.isclose(iqr, 0.0):
117                     continue
118
119                 lower = q1 - factor * iqr
120                 upper = q3 + factor * iqr
121                 outlier_mask = (~np.isfinite(values)) | (values < lower) | (values >
122                     upper)
123                 normal_positions = np.flatnonzero(~outlier_mask)

```

```

120
121     for position in np.flatnonzero(outlier_mask):
122         left_candidates = normal_positions[normal_positions < position]
123         right_candidates = normal_positions[normal_positions > position]
124         neighbors: list[float] = []
125         if left_candidates.size:
126             neighbors.append(float(values[left_candidates[-1]]))
127         if right_candidates.size:
128             neighbors.append(float(values[right_candidates[0]]))
129         if not neighbors:
130             continue
131
132         replacement = float(np.mean(neighbors))
133         row_index = group.index[position]
134         cleaned.at[row_index, column] = replacement
135         audit.append(
136             {
137                 "battery_id": int(battery_id),
138                 "cycle": int(group.iloc[position]["cycle"]),
139                 "column": column,
140                 "original": float(values[position]),
141                 "replacement": replacement,
142                 "lower_bound": float(lower),
143                 "upper_bound": float(upper),
144             }
145         )
146
147     return cleaned.sort_values(["battery_id", "cycle"]), audit
148
149
150 def build_policy_table(summary: pd.DataFrame) -> pd.DataFrame:
151     return (
152         summary[["battery_id", "policy"]]
153         .sort_values("battery_id")
154         .reset_index(drop=True)
155     )
156
157
158 def build_cycle_200_summary(
159     cleaned_cycles: pd.DataFrame,
160     policy_table: pd.DataFrame,
161 ) -> pd.DataFrame:
162     max_cycle = cleaned_cycles.groupby("battery_id")["cycle"].max()
163     eligible_ids = max_cycle[max_cycle == 200].index

```



```

164     eligible = cleaned_cycles[cleaned_cycles["battery_id"].isin(eligible_ids)]
165
166     cycle_200 = eligible.loc[eligible["cycle"] == 200, ["battery_id", "SOH"]]
167     if cycle_200["battery_id"].duplicated().any() or len(cycle_200) != len(
168         eligible_ids):
169         raise ValueError("达到 200 次循环的电池没有且仅有一条 cycle=200 记录")
170
171     charge_mean = (
172         eligible.groupby("battery_id", as_index=False)["chargetime"]
173         .mean()
174         .rename(columns={"chargetime": "charge_time_mean"})
175     )
176     result = (
177         cycle_200.merge(policy_table, on="battery_id", validate="one_to_one")
178         .merge(charge_mean, on="battery_id", validate="one_to_one")
179         [["battery_id", "policy", "SOH", "charge_time_mean"]]
180         .sort_values("battery_id")
181         .reset_index(drop=True)
182     )
183     return result
184
185 def build_policy_summary(
186     data: pd.DataFrame,
187     policy_order: list[str],
188     critical_soh_low: float,
189     critical_soh_high: float,
190     summary: pd.DataFrame,
191     cleaned_cycles: pd.DataFrame,
192 ) -> pd.DataFrame:
193     """汇总统一策略的最终 SOH、充电时间和其他实验参数。"""
194     classified = data.assign(
195         lifetime_group=np.select(
196             [data["SOH"] >= critical_soh_high, data["SOH"] <= critical_soh_low],
197             ["长寿命", "短寿命"],
198             default="中等寿命",
199         )
200     )
201     grouped = classified.groupby("policy", sort=False)
202     result = grouped.agg(
203         battery_count=("battery_id", "size"),
204         SOH_min=("SOH", "min"),
205         SOH_max=("SOH", "max"),
206         SOH_mean=("SOH", "mean"),

```

```

207     SOH_median=("SOH", "median"),
208     charge_time_mean=("charge_time_mean", "mean"),
209     charge_time_min=("charge_time_mean", "min"),
210     charge_time_max=("charge_time_mean", "max"),
211 ).reindex(policy_order)
212 result["battery_ids"] = grouped["battery_id"].apply(
213     lambda values: ",".join(str(int(value)) for value in sorted(values))
214 ).reindex(policy_order)
215
216 eligible_ids = set(int(value) for value in data["battery_id"])
217 battery_cycle_metrics = (
218     cleaned_cycles.loc[cleaned_cycles["battery_id"].isin(eligible_ids)]
219     .groupby(["battery_id", "policy"], as_index=False)
220     .agg(IR_mean=("IR", "mean"), Tavg_mean=("Tavg", "mean"))
221 )
222 cycle_metrics = battery_cycle_metrics.groupby("policy").agg(
223     IR_mean=("IR_mean", "mean"),
224     Tavg_mean=("Tavg_mean", "mean"),
225 ).reindex(policy_order)
226 result = result.join(cycle_metrics)
227
228 eligible_summary = summary.loc[summary["battery_id"].isin(eligible_ids)]
229 capacity_metrics = eligible_summary.groupby("policy").agg(
230     initial_capacity_mean=("initial_capacity", "mean"),
231     initial_capacity_min=("initial_capacity", "min"),
232     initial_capacity_max=("initial_capacity", "max"),
233 ).reindex(policy_order)
234 result = result.join(capacity_metrics)
235
236 def parameter_values(series: pd.Series) -> str:
237     values = [f"{float(value):g}" for value in sorted(series.dropna().unique
238         ())]
239     if series.isna().any():
240         values.append("缺失")
241     return "/".join(values)
242
243 for column in ["C1", "Q1", "C2"]:
244     if column in eligible_summary.columns:
245         result[f"{column}_values"] = (
246             eligible_summary.groupby("policy")[column].apply(parameter_values)
247             .reindex(policy_order)
248         )
249 group_counts = (
250     classified.groupby(["policy", "lifetime_group"])

```

```

249         .size()
250         .unstack(fill_value=0)
251         .reindex(policy_order, fill_value=0)
252     )
253     for group_name, column_name in [
254         ("长寿命", "long_battery_count"),
255         ("中等寿命", "middle_battery_count"),
256         ("短寿命", "short_battery_count"),
257     ]:
258         result[column_name] = group_counts.get(group_name, pd.Series(0, index=
                result.index)).astype(int)
259
260     result["lifetime_label"] = np.select(
261         [result["SOH_median"] >= critical_soh_high, result["SOH_median"] <=
                critical_soh_low],
262         ["长寿命策略", "短寿命策略"],
263         default="中等寿命策略",
264     )
265     max_median = result["SOH_median"].max()
266     min_median = result["SOH_median"].min()
267     result["longest_strategy"] = np.where(np.isclose(result["SOH_median"],
                max_median), "是", "否")
268     result["shortest_strategy"] = np.where(np.isclose(result["SOH_median"],
                min_median), "是", "否")
269     result["SOH_range"] = result.apply(lambda row: f"{row['SOH_min']:.6f} - {row
                ['SOH_max']:.6f}", axis=1)
270     result["charge_time_range"] = result.apply(
271         lambda row: f"{row['charge_time_min']:.6f} - {row['charge_time_max']:.6f}"
                , axis=1
272     )
273     return result.reset_index()
274
275
276 def save_soh_histogram(
277     data: pd.DataFrame,
278     output_path: Path,
279     critical_soh_low: float,
280     critical_soh_high: float,
281 ) -> None:
282     values = data["SOH"].to_numpy(dtype=float)
283     bin_edges = np.histogram_bin_edges(values, bins="fd")
284     if len(bin_edges) < 7:
285         bin_edges = np.linspace(values.min(), values.max(), 8)
286

```

```

287 fig, ax = plt.subplots(figsize=(10, 6))
288 ax.hist(
289     values,
290     bins=bin_edges,
291     color="#4C78A8",
292     edgecolor="white",
293     linewidth=1.0,
294     label="电池数量",
295 )
296 ax.axvline(
297     critical_soh_high,
298     color="#D62728",
299     linestyle="--",
300     linewidth=2,
301     label=f"上四分位线 (Q3) = {critical_soh_high:.4f}",
302 )
303 ax.axvline(
304     critical_soh_low,
305     color="#1F77B4",
306     linestyle="--",
307     linewidth=2,
308     label=f"下四分位线 (Q1) = {critical_soh_low:.4f}",
309 )
310 margin = max((values.max() - values.min()) * 0.08, 0.003)
311 ax.set_xlim(values.min() - margin, values.max() + margin)
312 ax.set_title("第 200 次循环 SOH 分布")
313 ax.set_xlabel("第 200 次循环的 SOH")
314 ax.set_ylabel("电池数量")
315 ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
316 ax.legend(loc="upper right")
317 fig.tight_layout()
318 fig.savefig(output_path, dpi=300, bbox_inches="tight")
319 plt.close(fig)
320
321
322 def save_soh_charge_scatter(
323     data: pd.DataFrame,
324     output_path: Path,
325     policy_order: list[str],
326 ) -> None:
327     fig, ax = plt.subplots(figsize=(13, 8))
328     color_map = matplotlib.colormaps.get_cmap("tab10")
329     colors = color_map(np.linspace(0, 1, max(len(policy_order), 2)))
330     for color, policy in zip(colors, policy_order):

```

```

331     group = data.loc[data["policy"] == policy]
332     if group.empty:
333         continue
334     ax.scatter(
335         group["SOH"],
336         group["charge_time_mean"],
337         s=64,
338         color=color,
339         edgecolors="white",
340         linewidths=0.8,
341         alpha=0.88,
342         label=policy,
343     )
344     ax.set_title("第 200 次循环 SOH 与平均充电时间")
345     ax.set_xlabel("第 200 次循环的 SOH")
346     ax.set_ylabel("平均充电时间 (min) ")
347     ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
348     ax.legend(loc="upper right", fontsize=8, framealpha=0.92, title="充电策略",
349             title_fontsize=9)
350     fig.tight_layout()
351     fig.savefig(output_path, dpi=300, bbox_inches="tight")
352     plt.close(fig)
353
354 def save_policy_boxplot(
355     data: pd.DataFrame,
356     output_path: Path,
357     critical_soh_low: float,
358     critical_soh_high: float,
359     policy_order: list[str],
360 ) -> None:
361     available_order = [policy for policy in policy_order if policy in set(data["
362         policy"])]
363     groups = [data.loc[data["policy"] == policy, "SOH"].to_numpy() for policy in
364         available_order]
365
366     fig, ax = plt.subplots(figsize=(16, 8))
367     box = ax.boxplot(
368         groups,
369         tick_labels=available_order,
370         patch_artist=True,
371         widths=0.62,
372         medianprops={"color": "#1F1F1F", "linewidth": 1.5},
373         whiskerprops={"color": "#555555"},

```

```

372     capprops={"color": "#555555"},
373     flierprops={
374         "marker": "o",
375         "markerfacecolor": "white",
376         "markeredgecolor": "#4C78A8",
377         "markersize": 5,
378     },
379 )
380 for patch in box["boxes"]:
381     patch.set_facecolor("#9ECAE1")
382     patch.set_edgecolor("#4C78A8")
383
384 ax.axhline(
385     critical_soh_high,
386     color="#D62728",
387     linestyle="--",
388     linewidth=2,
389 )
390 ax.axhline(
391     critical_soh_low,
392     color="#1F77B4",
393     linestyle="--",
394     linewidth=2,
395 )
396 all_values = data["SOH"].to_numpy(dtype=float)
397 margin = max((all_values.max() - all_values.min()) * 0.08, 0.003)
398 ax.set_ylim(all_values.min() - margin, all_values.max() + margin)
399 ax.set_title("不同充电策略下的第 200 次循环 SOH 分布")
400 ax.set_xlabel("充电策略")
401 ax.set_ylabel("使用寿命指标: 第 200 次循环的 SOH")
402 ax.tick_params(axis="x", labelrotation=40)
403 for label in ax.get_xticklabels():
404     label.set_horizontalalignment("right")
405 ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
406 legend_handles = [
407     Patch(facecolor="#9ECAE1", edgecolor="#4C78A8", label="策略内 SOH 分布"),
408     plt.Line2D(
409         [0], [0], color="#D62728", linestyle="--", linewidth=2,
410         label=f"上四分位线 (Q3)={critical_soh_high:.4f}",
411     ),
412     plt.Line2D(
413         [0], [0], color="#1F77B4", linestyle="--", linewidth=2,
414         label=f"下四分位线 (Q1)={critical_soh_low:.4f}",
415     ),

```

```

416 ]
417 ax.legend(handles=legend_handles, loc="upper right")
418 fig.tight_layout()
419 fig.savefig(output_path, dpi=300, bbox_inches="tight")
420 plt.close(fig)
421
422
423 def write_typical_policy_doc(
424     doc_path: Path,
425     policy_summary: pd.DataFrame,
426     critical_soh_low: float,
427     critical_soh_high: float,
428 ) -> None:
429     long_policies = policy_summary.loc[policy_summary["lifetime_label"] == "长寿命策略"]
430     short_policies = policy_summary.loc[policy_summary["lifetime_label"] == "短寿命策略"]
431     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是"].iloc[0]
432     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是"].iloc[0]
433
434     lines = [
435         "# 典型长寿命与短寿命充电策略",
436         "",
437         "本分析使用第 200 次循环的 SOH 作为早期使用寿命代理指标。",
438         f"长寿命阈值为样本 SOH 的 75% 分位数 ({critical_soh_high:.6f}) , ",
439         f"短寿命阈值为 25% 分位数 ({critical_soh_low:.6f})。策略类型按策略内 SOH 中位数判定。",
440         "",
441         "## 关键结论",
442         "",
443         f"- **寿命最长策略: `{longest['policy']}`**, SOH 中位数 {longest['SOH_median']:.6f}, 范围 {longest['SOH_range']}。",
444         f"- **寿命最短策略: `{shortest['policy']}`**, SOH 中位数 {shortest['SOH_median']:.6f}, 范围 {shortest['SOH_range']}。",
445         "",
446         "## 典型长寿命策略",
447         "",
448         "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 |",
449         "| ---| ---: | ---: | ---| ---| ---|",
450     ]
451     for _, row in long_policies.sort_values("SOH_median", ascending=False).iterrows():

```

```

452     mark = "寿命最长" if row["longest_strategy"] == "是" else "典型长寿命"
453     lines.append(
454         f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
455         f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** | "
456     )
457     lines.extend(
458         [
459             "",
460             "## 典型短寿命策略",
461             "",
462             "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 | "
463             ,
464             "| ---|---:|---:|---|---|---|",
465         ]
466     )
467     for _, row in short_policies.sort_values("SOH_median").iterrows():
468         mark = "寿命最短" if row["shortest_strategy"] == "是" else "典型短寿命"
469         lines.append(
470             f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
471             f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** | "
472         )
473         lines.extend(
474             [
475                 "",
476                 "## 解释限制",
477                 "",
478                 "- 各策略的完整 200 循环样本数较少，结论用于本数据集内的典型策略比较，不宜直接外推为因果结论。",
479                 "- `4_8C_80PER_4_8C_NEWSTRUCTURE` 的中位数较高但包含一个明显较低的 SOH 个体，需结合离散程度解读。",
480                 "- 数据未覆盖 SOH<80% 的真实寿命终止循环，因此这里比较的是第 200 次循环的健康保持程度。",
481             ]
482         )
483     doc_path.parent.mkdir(parents=True, exist_ok=True)
484     doc_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
485
486 def write_policy_report(
487     report_path: Path,
488     policy_summary: pd.DataFrame,
489     critical_soh_low: float,

```



```

490     critical_soh_high: float,
491 ) -> None:
492     """输出第三次迭代要求的统一策略统计报告。"""
493     lines = [
494         "# 统一充电策略统计报告",
495         "",
496         "## 统计口径",
497         "",
498         "- 策略名称已按第三次迭代规则合并，并移除 `_NEWSTRUCTURE` 后缀。",
499         "- 最终 SOH 为完整记录至第 200 次循环的电池在 `cycle=200` 时的清洗后 SOH。"
500         ,
501         "- 充电时间、内阻和温度先按电池对 1-200 次循环取均值，再按策略汇总。",
502         f"- 下四分位线 Q1={critical_soh_low:.6f}; 上四分位线 Q3={critical_soh_high:.6f}。",
503         "",
504         "## 最终 SOH 与充电时间",
505         "",
506         "| 统一策略 | 电池记号 | 电池数 | 最终SOH平均 | 最小 | 最大 | 充电时间平均/"
507         "min | 最小/min | 最大/min |",
508         "|---|---|---:|---:|---:|---:|---:|---:|",
509     ]
510     for _, row in policy_summary.iterrows():
511         lines.append(
512             f"| `{row['policy']}` | {row['battery_ids']} | {int(row['battery_count'])} | "
513             f"{row['SOH_mean']:.6f} | {row['SOH_min']:.6f} | {row['SOH_max']:.6f} | "
514             f"{row['charge_time_mean']:.6f} | {row['charge_time_min']:.6f} | {row['charge_time_max']:.6f} | "
515         )
516     lines.extend(
517         [
518             "",
519             "## 其他参数",
520             "",
521             "| 统一策略 | C1取值 | Q1取值 | C2取值 | 初始容量平均/Ah | 最小/Ah | 最大/Ah | 平均内阻 | 平均温度/°C |",
522             "|---|---|---|---:|---:|---:|---:|---:|",
523         ]
524     )
525     for _, row in policy_summary.iterrows():
526         lines.append(
527             f"| `{row['policy']}` | {row.get('C1_values', '')} | {row.get('Q1_values', '')} | "

```

```

526         f"{row.get('C2_values', '')} | {row['initial_capacity_mean']:.6f} | "
527         f"{row['initial_capacity_min']:.6f} | {row['initial_capacity_max']:.6f} | "
528         f"{row['IR_mean']:.6f} | {row['Tavg_mean']:.3f} | "
529     )
530     lines.extend(
531         [
532             "",
533             "## 解释限制",
534             "",
535             "- 9块仅有150次循环数据的测试电池未纳入最终SOH策略统计。",
536             "- 统一策略是按题目指定规则进行的分析分组；报告中的相关性不代表充电策略对寿命的因果效应。",
537             "- 本数据片段尚未覆盖 SOH<80% 的真实寿命终止点，最终SOH用于比较第200次循环时的健康保持程度。",
538         ]
539     )
540     report_path.parent.mkdir(parents=True, exist_ok=True)
541     report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
542
543
544 def build_acceptance_results(
545     original_summary: pd.DataFrame,
546     normalized_summary: pd.DataFrame,
547     normalized_cycles: pd.DataFrame,
548     policy_table: pd.DataFrame,
549     cycle_200: pd.DataFrame,
550     policy_summary: pd.DataFrame,
551     soh_dir: Path,
552     table_write_paths: dict[str, Path],
553 ) -> list[tuple[str, bool, str]]:
554     """执行第三次迭代的显式验收检查。"""
555     original_count = int(original_summary["policy"].nunique())
556     normalized_count = int(normalized_summary["policy"].nunique())
557     expected_names = {normalize_policy_name(value) for value in original_summary["policy"].unique()}
558     actual_names = set(normalized_summary["policy"].unique())
559     forbidden = {"8OPER_3_6C", "4_8C_8OPER_4_8C_NEWSTRUCTURE"}
560     plot_files = list(soh_dir.glob("SOH*.png"))
561     individual_files = [path for path in plot_files if path.stem != "SOH_ALL"]
562     results = [
563         (
564             "策略数量恰好减少2种",
565             original_count - normalized_count == 2,

```

```

566         f"原始{original_count}种，统一后{normalized_count}种，减少{
567             original_count - normalized_count}种",
568     ),
569     (
570         "统一策略集合正确",
571         actual_names == expected_names,
572         f"实际策略: {sorted(actual_names)}",
573     ),
574     (
575         "旧策略名称已消除",
576         not (actual_names & forbidden),
577         f"禁止名称残留: {sorted(actual_names & forbidden)}",
578     ),
579     (
580         "NEWSTRUCTURE后缀已全部移除",
581         not any("NEWSTRUCTURE" in name.upper() for name in actual_names),
582         "统一后的策略名不含NEWSTRUCTURE",
583     ),
584     (
585         "全部处理表命名一致",
586         set(normalized_cycles["policy"].unique()) == actual_names
587         and set(policy_table["policy"].unique()) == actual_names
588         and set(cycle_200["policy"].unique()).issubset(actual_names),
589         "清洗循环表、charge_policy和battery_SOH_charge均采用统一名称",
590     ),
591     (
592         "charge_policy覆盖49块电池",
593         len(policy_table) == 49
594         and policy_table["battery_id"].nunique() == 49,
595         f"记录数={len(policy_table)}, 唯一电池数={policy_table['battery_id'].
596             nunique()}",
597     ),
598     (
599         "核心CSV均写入标准文件名",
600         all(path.name == expected for expected, path in table_write_paths.
601             items()),
602         ", ".join(f"{expected}->{path.name}" for expected, path in
603             table_write_paths.items()),
604     ),
605     (
606         "策略报告覆盖全部统一策略",
607         set(policy_summary["policy"]) == actual_names and len(policy_summary)
608             == normalized_count,
609         f"报告策略数={len(policy_summary)}",

```

```

605     ),
606     (
607         "49张单体曲线与SOH_ALL齐全",
608         len(individual_files) == 49 and (soh_dir / "SOH_ALL.png").is_file(),
609         f"单体曲线={len(individual_files)}, SOH_ALL={int((soh_dir / 'SOH_ALL.
        png').is_file())}",
610     ),
611 ]
612 return results
613
614
615 def write_acceptance_report(
616     path: Path,
617     results: list[tuple[str, bool, str]],
618     mapping: dict[str, str],
619 ) -> None:
620     lines = [
621         "# 第三次迭代验收确认",
622         "",
623         "| 验收项 | 结果 | 证据 |",
624         "|---|---|---|",
625     ]
626     for name, passed, evidence in results:
627         lines.append(f"| {name} | {'通过' if passed else '失败'} | {evidence} |")
628     lines.extend(["", "## 策略名称映射", "", "| 原始名称 | 统一名称 |", "
        |---|---|"])
629     for source, target in mapping.items():
630         lines.append(f"| `{source}` | `{target}` |")
631     lines.extend(["", f"**总体结论: {'全部通过' if all(item[1] for item in
        results) else '存在失败项'}.**"])
632     path.parent.mkdir(parents=True, exist_ok=True)
633     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
634
635
636 def write_csv_if_changed(data: pd.DataFrame, path: Path) -> Path:
637     """内容不变时避免重写，兼容 CSV 正被表格软件打开的情形。"""
638     if path.exists():
639         try:
640             existing = pd.read_csv(path)
641             pd.testing.assert_frame_equal(existing, data, check_dtype=False)
642             return path
643         except (AssertionError, OSError, ValueError):
644             pass
645     pending_path = path.with_name(f"{path.stem}_v3_pending{path.suffix}")

```

```

646     try:
647         data.to_csv(path, index=False, encoding="utf-8-sig")
648         if pending_path.exists():
649             pending_path.unlink()
650         return path
651     except PermissionError:
652         data.to_csv(pending_path, index=False, encoding="utf-8-sig")
653         return pending_path
654
655
656 def write_log(
657     log_path: Path,
658     summary: pd.DataFrame,
659     cycles: pd.DataFrame,
660     audit: list[dict[str, float | int | str]],
661     cycle_200: pd.DataFrame,
662     policy_summary: pd.DataFrame,
663     critical_soh_low: float,
664     critical_soh_high: float,
665     iqr_factor: float,
666     font_name: str,
667     output_paths: list[Path],
668     prompt_text: str,
669     policy_mapping: dict[str, str],
670     acceptance_results: list[tuple[str, bool, str]],
671 ) -> None:
672     counts = Counter(str(item["column"]) for item in audit)
673     max_cycle_counts = cycles.groupby("battery_id")["cycle"].max().value_counts
674         ().sort_index()
675     missing_summary = int(summary.isna().sum().sum())
676     missing_cycles = int(cycles.isna().sum().sum())
677     long_count = int((cycle_200["SOH"] >= critical_soh_high).sum())
678     short_count = int((cycle_200["SOH"] <= critical_soh_low).sum())
679     middle_count = len(cycle_200) - long_count - short_count
680     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是", "
        policy"].iloc[0]
681     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是", "
        policy"].iloc[0]
682     normalized_policy_count = len(set(policy_mapping.values()))
683
684     lines = [
685         "# 问题一第三次迭代运行日志 (log_3) ",
686         "",
687         f"- 运行时间: {datetime.now().astimezone().isoformat(timespec='seconds')}

```

```

        ",
687 f"- IQR 系数: {iqr_factor:.6g}",
688 f"- 上四分位线 Q3 (75%分位数): {critical_soh_high:.6f}",
689 f"- 下四分位线 Q1 (25%分位数): {critical_soh_low:.6f}",
690 f"- 绘图字体: {font_name}",
691 "",
692 "## 输入质量检查",
693 "",
694 f"- battery_summary.csv: {len(summary)} 行, {summary.shape[1]} 列, {
        summary['battery_id'].nunique()} 块电池。",
695 f"- cycle_train.csv: {len(cycles)} 行, {cycles.shape[1]} 列。",
696 f"- battery_id + cycle 重复记录: {int(cycles.duplicated(['battery_id', '
        cycle']).sum())}。",
697 f"- 汇总表缺失单元格: {missing_summary} (其中 C1 的结构性缺失保留, 不参与循
        环数据清洗)。",
698 f"- 循环表缺失单元格: {missing_cycles}。",
699 f"- 最大循环数分布: {dict((int(k), int(v)) for k, v in max_cycle_counts.
        items())}。",
700 "",
701 "## 策略统一命名",
702 "",
703 f"- 原始策略数: {len(policy_mapping)}; 统一后策略数: {
        normalized_policy_count}; 恰好减少 {len(policy_mapping) -
        normalized_policy_count} 种。",
704 "- `8OPER_3_6C` 统一为 `3_6C-8OPER_3_6C`。",
705 "- `4_8C_8OPER_4_8C_NEWSTRUCTURE` 统一为 `4_8C_8OPER_4_8C`。",
706 "- 其余策略仅移除 `_NEWSTRUCTURE` 后缀。",
707 "",
708 "## IQR 清洗",
709 "",
710 "- 方法: 按 battery_id 分组, 对 capacity、SOH、SOH_smooth、chargetime、IR
        、Tavg 分别计算 Q1、Q3 和 IQR。",
711 "- 判定: 小于 Q1 - 1.5×IQR 或大于 Q3 + 1.5×IQR 的值标记为异常。",
712 "- 填补: 使用同一电池、同一字段中距离最近的前后正常值均值; 边界点仅有一侧正
        常值时使用该侧值。",
713 f"- 共替换 {len(audit)} 个单元格; 分字段数量: {dict(counts)}。",
714 "- 原始 data 文件未改写; 清洗结果另存为 output/A/cycle_train_cleaned.csv。
        ",
715 "",
716 "## 200 次循环样本",
717 "",
718 f"- 纳入: 最大 cycle 恰为 200 的 {len(cycle_200)} 块电池。",
719 f"- 排除: 仅记录至 150 次循环的 {summary['prediction_test'].sum()} 块测试
        电池。",

```

```

720 f"- SOH 范围: {cycle_200['SOH'].min():.6f} - {cycle_200['SOH'].max():.6f}
    , 均值 {cycle_200['SOH'].mean():.6f}。",
721 f"- 按两个分位数分类: 长寿命 {long_count} 块, 中等寿命 {middle_count} 块,
    短寿命 {short_count} 块。",
722 f"- 寿命最长策略 (按策略 SOH 中位数): {longest}。",
723 f"- 寿命最短策略 (按策略 SOH 中位数): {shortest}。",
724 "",
725 "## SOH 曲线",
726 "",
727 "- 使用清洗后的 `SOH_smooth` 绘制49张单体曲线和1张全部电池曲线。",
728 "- 单体曲线命名为 `SOH_N.png`; 总图命名为 `SOH_ALL.png`, 颜色按电池编号由蓝
    到红渐变。",
729 "",
730 "## 验收确认",
731 "",
732 ]
733 for name, passed, evidence in acceptance_results:
734     lines.append(f"- [{ 'x' if passed else ' '}] {name}: {evidence}。")
735 lines.extend(["", "## 输出文件", ""])
736 lines.extend(f"- `{path.relative_to(PROJECT_ROOT).as_posix()}`" for path in
    output_paths)
737 lines.extend(
738     [
739         "",
740         "## 说明",
741         "",
742         "- 箱型图纵轴采用“第 200 次循环 SOH”作为使用寿命代理指标; 本题数据尚未
            覆盖 SOH<80% 的真实寿命终止循环。",
743         "- IQR 是统计异常筛查, 不等同于确认传感器故障; 清洗结果用于完成本题指定
            分析。",
744         "- 图中的两条参考线统一命名为上四分位线 (Q3) 和下四分位线 (Q1)。",
745         "- 策略报告使用统一后的7种策略, 最终SOH统计只纳入完整200循环的40块电池。
            ",
746         "",
747         "## 原始提示词",
748         "",
749         "以下完整保留本次执行时 `A/问题一/AGENTS.md` 的内容: ",
750         "",
751         "``text",
752         prompt_text.rstrip(),
753         "``",
754     ]
755 )
756 log_path.parent.mkdir(parents=True, exist_ok=True)

```

```

757     log_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
758
759
760 def parse_args() -> argparse.Namespace:
761     parser = argparse.ArgumentParser(description="运行问题一完整分析流程")
762     parser.add_argument("--iqr-factor", type=float, default=1.5, help="IQR 异常
763         判定系数, 默认 1.5")
764     return parser.parse_args()
765
766 def main() -> None:
767     args = parse_args()
768     if args.iqr_factor <= 0:
769         raise ValueError("--iqr-factor 必须大于 0")
770
771     data_dir = PROJECT_ROOT / "data"
772     output_dir = PROJECT_ROOT / "output" / "A"
773     output_dir.mkdir(parents=True, exist_ok=True)
774     original_summary = pd.read_csv(data_dir / "battery_summary.csv")
775     original_cycles = pd.read_csv(data_dir / "cycle_train.csv")
776     validate_inputs(original_summary, original_cycles)
777     summary, cycles, policy_mapping = normalize_policies(original_summary,
778         original_cycles)
779     validate_inputs(summary, cycles)
780
781     cleaned, audit = iqr_clean_by_battery(cycles, MEASUREMENT_COLUMNS, args.
782         iqr_factor)
783     policy_table = build_policy_table(summary)
784     cycle_200 = build_cycle_200_summary(cleaned, policy_table)
785     critical_soh_low = float(cycle_200["SOH"].quantile(0.25))
786     critical_soh_high = float(cycle_200["SOH"].quantile(0.75))
787     policy_order = list(dict.fromkeys(summary.sort_values("battery_id")["policy"]
788         ))
789     policy_summary = build_policy_summary(
790         cycle_200,
791         policy_order,
792         critical_soh_low,
793         critical_soh_high,
794         summary,
795         cleaned,
796     )
797
798     cleaned_path = output_dir / "cycle_train_cleaned.csv"
799     policy_path = output_dir / "charge_policy.csv"

```



```

797 summary_path = output_dir / "battery_SOH_charge.csv"
798 soh_example_path = output_dir / "SOH_1_example.png"
799 histogram_path = output_dir / "SOH_summary.png"
800 scatter_path = output_dir / "SOH_chargetime.png"
801 boxplot_path = output_dir / "charge_policy.png"
802 policy_summary_path = output_dir / "policy.csv"
803 report_path = PROJECT_ROOT / "output" / "doc" / "1.md"
804 acceptance_path = output_dir / "acceptance_3.md"
805 soh_dir = PROJECT_ROOT / "A" / "问题一" / "SOH"
806
807 cleaned_write_path = write_csv_if_changed(cleaned, cleaned_path)
808 policy_write_path = write_csv_if_changed(policy_table, policy_path)
809 summary_write_path = write_csv_if_changed(cycle_200, summary_path)
810 policy_summary_write_path = write_csv_if_changed(policy_summary,
811         policy_summary_path)
812
813 font_name = configure_chinese_font()
814 plot_battery_soh(cleaned, 1, soh_example_path)
815 save_soh_histogram(cycle_200, histogram_path, critical_soh_low,
816         critical_soh_high)
817 save_soh_charge_scatter(cycle_200, scatter_path, policy_order)
818 save_policy_boxplot(
819     cycle_200,
820     boxplot_path,
821     critical_soh_low,
822     critical_soh_high,
823     policy_order,
824 )
825 write_policy_report(
826     report_path,
827     policy_summary,
828     critical_soh_low,
829     critical_soh_high,
830 )
831
832 soh_dir.mkdir(parents=True, exist_ok=True)
833 for stale_plot in soh_dir.glob("SOH_*.png"):
834     stale_plot.unlink()
835 for battery_id in sorted(int(value) for value in cleaned["battery_id"].
836     unique()):
837     plot_battery_soh(cleaned, battery_id, soh_dir / f"SOH_{battery_id}.png")
838 plot_all_battery_soh(cleaned, soh_dir / "SOH_ALL.png")
839
840 acceptance_results = build_acceptance_results(

```

```

838     original_summary,
839     summary,
840     cleaned,
841     policy_table,
842     cycle_200,
843     policy_summary,
844     soh_dir,
845     {
846         cleaned_path.name: cleaned_write_path,
847         policy_path.name: policy_write_path,
848         summary_path.name: summary_write_path,
849         policy_summary_path.name: policy_summary_write_path,
850     },
851 )
852 write_acceptance_report(acceptance_path, acceptance_results, policy_mapping)
853
854 output_paths = [
855     cleaned_write_path,
856     policy_write_path,
857     summary_write_path,
858     soh_example_path,
859     histogram_path,
860     scatter_path,
861     boxplot_path,
862     policy_summary_write_path,
863     report_path,
864     acceptance_path,
865     soh_dir,
866 ]
867 write_log(
868     PROJECT_ROOT / "A" / "问题一" / "log_3.md",
869     summary,
870     cycles,
871     audit,
872     cycle_200,
873     policy_summary,
874     critical_soh_low,
875     critical_soh_high,
876     args.iqr_factor,
877     font_name,
878     output_paths,
879     (PROJECT_ROOT / "A" / "问题一" / "AGENTS.md").read_text(encoding="utf-8")
880     ,
881     policy_mapping,

```

```

881         acceptance_results,
882     )
883
884     print(f"IQR 替换单元格数: {len(audit)}")
885     print(f"200 次循环样本数: {len(cycle_200)}")
886     print(f"critical_SOH_low: {critical_soh_low:.6f}")
887     print(f"critical_SOH_high: {critical_soh_high:.6f}")
888     print(f"策略数量: {len(policy_mapping)} -> {len(set(policy_mapping.values()))}")
889     print(f"SOH曲线数量: {len(list(soh_dir.glob('SOH_*.png')))}")
890     failed = [name for name, passed, _ in acceptance_results if not passed]
891     print(f"验收失败项: {failed}")
892     for path in output_paths:
893         print(f"已生成: {path}")
894
895
896 if __name__ == "__main__":
897     main()

```

C AI 工具使用声明

根据赛事论文格式规范，现就人工智能工具使用情况作如下声明。全文不出现参赛者姓名与学校信息。

C.1 所用工具

- OpenAI ChatGPT（记录版本：1.2026.190，模型 gpt5.6-sol-high）
- OpenAI Codex CLI（记录版本：v0.147.0，模型 gpt5.6-sol-high；DeepSeek-V4-Flash-0731）
- 对话式编程助手（用于文献整理、问题一结果写入 LaTeX 论文框架）

C.2 使用目的与环节

1. 讨论与协作：询问 Git 多人协作与仓库管理相关操作说明。
2. 问题一：依据人工撰写的 AGENTS.md 生成模块化分析代码，完成 IQR 清洗、策略统一（9 种合并为 7 种）、200 圈汇总、49 张单体曲线与总图；人工检查可运行性后调整参数并复现 log₃。
3. 文献：整理快充、SOH 预测与策略优化相关公开文献条目，写入 references/。

4. **论文**：将第三次迭代后的问题一成果写入 LaTeX 正文，并为问题二至四保留章节框架。

C.3 提示方式

主要方式为：先由人工完成题意分析与方法构想，再将任务写成 AGENTS.md 或结构化提示，交由 AI 生成代码或文档草稿，随后人工运行、核验数值与图件是否与数据一致。问题一提示词原文见 A/问题一/AGENTS.md 及 log_2.md 附录，此处不重复粘贴以免附录过长；支撑材料中保留全文。

C.4 采纳、修改与核验

- 问题一流水线经 log_3 与验收确认：IQR 替换 559 格、策略 9→7、200 圈样本 40 块、分位阈值 0.993433/0.996103、49 张单体曲线与 SOH_ALL 齐全，与正文一致。
 - 典型长/短寿命以 A/问题一/总结.md 为准：不以 policy.csv 中按中位数标记的 4.8C 组合并为“寿命最长”代表；因果表述保持为初步观察。
 - 问题二至四的模型尚未给出最终数值，正文相应部分明确标注为框架。
- AI 仅作为辅助工具，论文观点、模型选择与最终结论由参赛队负责。